

RoboCot - Autonomous Cotton Harvesting System

ME 429 Design Report - Sections 2.2 & 2.3

2.2 Overview of Possible Solutions

Autonomous cotton harvesting requires the integration of multiple subsystems spanning perception, motion planning and mechanical design. Rather than evaluating monolithic system configurations, this section adopts a component-based approach where each design decision point is analyzed independently. Since the RoboCot architecture is built on modular, interchangeable components, this approach allows systematic evaluation of alternatives at each decision point using consistent criteria. The selected components are then integrated into the final system configuration.

2.2.1 Solution Space Overview

The design of an autonomous cotton picking system involves ten key decision points spanning software architecture, perception, control and mechanical domains. Table 1 summarizes these decision points, their requirements derived from the product design specifications (Section 2.1) and their impact on overall system performance.

Table 1. Solution Space: Design Decision Points

Decision Point	Requirement Reference	Impact on System
Software Framework	MF-03 (modularity), RL-02 (crash recovery)	Determines integration architecture, debugging capability and code reusability
Camera Placement	QL-03 (90% detection), SR-02 (mock field coverage)	Affects viewing angles, occlusion handling and inspection distance
Depth Sensing Method	QL-01 ($\pm 5\text{mm}$ positioning), QL-02 ($\pm 3\text{mm}$ repeatability)	Determines 3D localization accuracy and calibration complexity
Detection Method	QL-03 (90% detection), EN-01 (lighting robustness)	Affects detection reliability under varying conditions
Cluster Identification	QL-05 (90% pick rate), RL-02 (software reliability)	Determines tracking accuracy across multiple scan positions
Clustering Algorithm	QL-05 (functional consistency)	Affects grouping correctness and outlier rejection
Motion Planning	SF-02 (speed limits), SF-03 (singularity handling)	Determines path safety, collision avoidance and execution reliability
Manipulator Configuration	SR-01 (520mm reach), WT-03 (tip deflection $< 10\text{mm}$)	Affects workspace coverage and approach trajectory flexibility

Decision Point	Requirement Reference	Impact on System
Gripper Design	SR-04 (50-60mm opening), QL-04 (cycle time <60s)	Determines grasp success rate and pick efficiency
Reservoir Design	SR-03 (15×15×15cm), ER-03 (tool-free removal <15s)	Affects storage capacity and operator interaction
Operator Interface	ER-01 (GUI control), ER-02 (visual indicators)	Determines operator situational awareness and control capability

The following subsections present the alternative options for each decision point, evaluate them against relevant criteria and justify the selected solution.

2.2.2 Design Decision Analysis

Decision Point 1: Software Framework

The software framework determines how system components communicate, how sensor data flows through the pipeline and how motion commands are executed. This choice affects development efficiency, debugging capability and long-term maintainability.

Option A: ROS2 (Robot Operating System 2) ROS2 is a widely adopted open-source framework for robotics development [18]. It provides standardized communication patterns (topics, services, actions), a transform library (TF2) for coordinate frame management and integration with common tools including Gazebo simulation and MoveIt motion planning.

Option B: Custom Framework A custom framework built on standard libraries (ZeroMQ, Protocol Buffers) offers more control over communication patterns and potentially lower latency. However, it requires implementing functionality that ROS2 provides out of the box.

Option C: NVIDIA Isaac ROS NVIDIA's Isaac ROS is a GPU-accelerated robotics framework optimized for Jetson platforms [31]. It provides hardware-accelerated perception pipelines and integrates with Isaac Sim for simulation. However, it has a smaller community and fewer manipulation-focused tools compared to the broader ROS ecosystem.

Table 2. Software Framework Decision Matrix

Criterion	Weight	PDS Ref.	ROS2	Custom	Isaac ROS
Tool Integration (MoveIt, Gazebo)	0.12	MF-03	5	2	3
Modularity & Reusability	0.12	MF-03	5	3	4
Development Speed	0.12	MF-02	4	2	3
Long-term Support	0.28	RL-02	5	3	4
Real-time Capability	0.36	QL-04	4	5	5
Weighted Total	1.00		4.52	3.48	4.12

Selected: ROS2 — Provides the best balance of tool integration, modularity and community support. While NVIDIA Isaac ROS offers superior GPU acceleration on the Jetson platform, MoveIt2 integration and the broader ecosystem of manipulation tools make ROS2 the preferred choice for this manipulation-focused application. The ros2_control framework enables standardized hardware abstraction while MoveIt2 provides motion planning capabilities essential for collision-free trajectory generation.

Decision Point 2: Camera Placement

Camera placement affects viewing angles, inspection distance and the ability to handle occlusions from plant geometry. For a wrist-mounted manipulator, two primary options exist.

Option A: Eye-in-Hand (Wrist-Mounted) The camera is mounted directly on the end-effector, moving with the gripper. This provides close-up inspection capability (viewing distance ~35cm) and the ability to position the camera at arbitrary viewpoints through arm motion.

Option B: Upper-Arm Mounted The camera is mounted on a more proximal link (shoulder or elbow region), providing a more stable platform but with less positioning flexibility and greater viewing distance to targets.

[Figure 1: Camera Placement Options - (a) Eye-in-hand configuration showing camera at wrist with close viewing distance, (b) Upper-arm mounted configuration showing camera on shoulder link with wider but less flexible viewing angle]

Table 3. Camera Placement Decision Matrix

Criterion	Weight	PDS Ref.	Eye-in-Hand	Upper-Arm
Close-up Inspection Capability	0.20	QL-03	5	2
Multi-angle Viewing	0.15	SR-02	5	3
Occlusion Handling	0.36	QL-03, EN-01	5	2
Platform Stability	0.20	QL-02	3	5
Calibration Simplicity	0.09	MA-03	3	4
Weighted Total	1.00		4.42	2.93

Selected: Eye-in-Hand — The ability to position the camera at multiple viewpoints and achieve close-up inspection is critical for reliable cotton detection and accurate depth measurement. The stability trade-off is acceptable given the controlled motion during scanning.

Decision Point 3: Depth Sensing Method

Accurate 3D localization requires depth measurement at detected pixel locations. Two primary approaches are available with the eye-in-hand configuration.

Option A: RGB-D Camera (Direct Depth) RGB-D cameras provide direct depth measurement at each pixel using structured light projection or time-of-flight sensing. The depth value is read directly from the sensor without additional computation.

Option B: Stereo Camera Pair (Triangulation) Stereo vision computes depth from disparity between matched features in left and right images. Depth accuracy depends on baseline distance and matching quality.

Table 4. Depth Sensing Decision Matrix

Criterion	Weight	PDS Ref.	RGB-D	Stereo
Depth Accuracy at Close Range	0.31	QL-01, QL-02	5	3
Calibration Simplicity	0.14	MA-03	5	2
Computational Cost	0.24	EN-02	5	3
Performance in Texture-less Regions	0.31	QL-03	4	2
Weighted Total	1.00		4.69	2.55

Selected: RGB-D Camera — Direct depth measurement eliminates stereo calibration complexity and matching errors. The ZED X Mini selected for hardware deployment provides depth accuracy compatible with the ±5mm positioning requirement at the 35cm viewing distance used during cluster inspection.

Decision Point 4: Detection Method

Cotton boll detection converts camera images into bounding boxes identifying target locations. This is the foundation of the perception pipeline.

Option A: Classical Computer Vision Traditional approaches use color segmentation (HSV thresholding for white cotton), edge detection and blob analysis. These methods are computationally efficient but sensitive to lighting variations.

Option B: Deep Learning (YOLO) Learning-based detection using convolutional neural networks trained on labeled cotton datasets. YOLO-style architectures provide real-time inference with strong robustness to lighting and background variation [12][22].

Table 5. Detection Method Decision Matrix

Criterion	Weight	PDS Ref.	Classical CV	YOLO
Robustness to Lighting Variation	0.18	EN-01	2	5
Detection Accuracy	0.23	QL-03	2	5
Handling Complex Backgrounds	0.41	QL-03, RL-01	2	5
Computational Cost	0.18	EN-02	5	3
Weighted Total	1.00		2.54	4.64

Selected: YOLO11 — Deep learning detection is essential for achieving the 90% detection accuracy requirement under field-like lighting conditions. The YOLO11 model trained on the Cotton-boll-and-cluster-2 dataset achieves confidence above 0.7 on cotton bolls. GPU inference on the Jetson Orin NX provides real-time performance.

Decision Point 5: Cluster Identification Strategy

During panoramic scanning, the same cotton cluster may be detected from multiple viewpoints. A method is required to identify that these detections correspond to the same physical target.

Option A: Multi-Frame Tracking (ByteTrack/BoT-SORT) Video-based trackers maintain object identity across frames using motion prediction and appearance features [25][26]. These methods are designed for continuous video streams where objects move smoothly between frames.

Option B: World-Space 3D Clustering (Custom Pipeline) Each detection is converted to 3D world coordinates using depth and TF transforms. Detections are then clustered based on spatial proximity in world-space, independent of camera motion between scan positions.

Table 6. Cluster Identification Decision Matrix

Criterion	Weight	PDS Ref.	ByteTrack	World-Space 3D
Stability Across Discrete Scan Positions	0.32	QL-05	2	5
ID Consistency	0.25	RL-02	3	5
Independence from Frame Rate	0.32	QL-04	2	5
Implementation Complexity	0.11	MF-03	4	3
Weighted Total	1.00		2.47	4.78

Selected: World-Space 3D Clustering — Testing with ByteTrack revealed ID instability when the camera moved significantly between scan positions, as the tracker's motion model assumes continuous video with gradual object movement. The world-space approach converts each detection to 3D coordinates and clusters spatially, providing robust identification regardless of camera trajectory.

Decision Point 6: Clustering Algorithm

When grouping detections in world-space, the algorithm choice affects how nearby detections are merged and whether chain-linking artifacts occur.

Option A: Single-Linkage Clustering A detection joins a cluster if it is within the merge radius of ANY existing member. This can cause chain-linking where distant detections become grouped through intermediate detections.

Option B: Complete-Linkage Clustering A detection joins a cluster only if it is within the merge radius of ALL existing members. This produces tight, compact clusters without chain-linking artifacts.

Option C: DBSCAN Density-based clustering identifies clusters as dense regions separated by sparse regions. Requires tuning of epsilon and minimum points parameters.

Table 7. Clustering Algorithm Decision Matrix

Criterion	Weight	PDS Ref.	Single-Link	Complete-Link	DBSCAN
Cluster Compactness	0.25	QL-05	2	5	4

Criterion	Weight	PDS Ref.	Single-Link	Complete-Link	DBSCAN
Resistance to Chain-Linking	0.46	QL-05, RL-02	1	5	4
Simplicity	0.09	MF-03	5	4	3
Parameter Sensitivity	0.20	RL-02	4	4	2
Weighted Total	1.00		2.21	4.71	3.51

Selected: Complete-Linkage Clustering — Initial testing with single-linkage produced chain-linking artifacts where separate clusters were incorrectly merged. Complete-linkage ensures that all members of a cluster are within the merge radius of each other, producing physically meaningful groupings. The merge radius of 0.121m (25% of minimum inter-cluster distance) ensures bolls on the same plant group together while separate plants remain distinct.

Decision Point 7: Motion Planning

Motion planning determines how the arm moves between positions and approaches targets while avoiding collisions and respecting joint limits.

Option A: Direct Joint Interpolation Joint angles are interpolated linearly between start and goal configurations. Simple to implement but provides no collision checking or Cartesian path control.

Option B: MoveIt2 with OMPL MoveIt2 provides a complete motion planning framework with collision checking, multiple planner options (OMPL library) and Cartesian path planning [19]. Integrates with ROS2 and ros2_control.

Option C: Custom IK Solver Inverse kinematics computed analytically or numerically for specific arm geometry. Provides direct control but requires custom collision checking implementation.

Table 8. Motion Planning Decision Matrix

Criterion	Weight	PDS Ref.	Joint Interp.	MoveIt2	Custom IK
Collision Avoidance	0.37	SF-02, SF-03	1	5	3
Cartesian Path Control	0.30	QL-01	1	5	4
Integration with ROS2	0.10	MF-03	3	5	2
Computational Overhead	0.23	EN-02	5	3	4
Weighted Total	1.00		2.12	4.54	3.43

Selected: MoveIt2 with OMPL — Collision avoidance is critical for safe operation in the cluttered mock field environment. MoveIt2 provides planning scene management, multiple planner algorithms and seamless integration with the ROS2 ecosystem. The computational overhead is acceptable given the non-time-critical nature of arm repositioning during scanning.

Decision Point 8: Manipulator Configuration

The manipulator must provide sufficient reach to cover the mock field while enabling approach trajectories that avoid plant obstacles.

Option A: 4-DOF Arm Four degrees of freedom (base rotation, shoulder, elbow, wrist) provide basic positioning capability but limit approach angle flexibility.

Option B: 6-DOF Arm (Braccio) Six degrees of freedom enable full position and orientation control of the end-effector, allowing the gripper to approach targets from arbitrary angles.

Table 9. Manipulator Configuration Decision Matrix

Criterion	Weight	PDS Ref.	4-DOF	6-DOF
Approach Trajectory Flexibility	0.29	QL-05	2	5
Workspace Coverage	0.23	SR-01, SR-02	3	4
Obstacle Avoidance Capability	0.35	SF-03	2	5
Mechanical Simplicity	0.13	MA-01	5	3
Weighted Total	1.00		2.62	4.51

Selected: 6-DOF Braccio Arm — The additional degrees of freedom enable approach trajectories that avoid plant obstacles while positioning the gripper optimally for cotton extraction. The Braccio arm provides 520mm+ reach satisfying requirement SR-01 and has existing ROS2/MoveIt integration from the EI_Pick_n_Place reference implementation.

Decision Point 9: Gripper Design

The gripper must reliably grasp cotton bolls without damaging the plant or losing the picked material during transfer.

Option A: Parallel Jaw Gripper Two opposing fingers close to grasp the target. Simple, reliable and provides controlled grip force. Requires 50-60mm opening for cotton boll clusters.

Option B: Vacuum/Suction End-Effector Suction cups or vacuum nozzles pick up material through negative pressure. Works well for flat surfaces but cotton's fibrous texture may reduce seal effectiveness.

Option C: Multi-Finger Adaptive Gripper Compliant fingers conform to irregular object shapes. Higher cost and complexity but better adaptation to variable boll geometry.

Table 10. Gripper Design Decision Matrix

Criterion	Weight	PDS Ref.	Parallel Jaw	Vacuum	Multi-Finger
Reliability on Fibrous Material	0.31	QL-05	4	2	4
Simplicity & Cost	0.28	MC-01, MF-01	5	4	2
Grip Security During Transfer	0.31	QL-05	4	2	5
3D Printability	0.10	MF-01	5	3	3

Criterion	Weight	PDS Ref.	Parallel Jaw	Vacuum	Multi-Finger
Weighted Total	1.00		4.38	2.66	3.65

Selected: Parallel Jaw Gripper — The parallel jaw design provides reliable grasping of cotton bolls with controlled force. The 50-60mm opening requirement (SR-04) is achievable with servo-driven fingers. Components can be 3D printed as specified in MF-01. The Braccio arm includes a compatible gripper base.

Decision Point 10: Reservoir Design

The reservoir collects harvested cotton bolls and must support easy emptying by the operator without tools.

Option A: Fixed Integrated Bin The collection bin is permanently attached to the platform. Cotton is removed by reaching into the bin or inverting the entire unit.

Option B: Removable Drop-In Bin A separate container drops into a receptacle and lifts out for emptying. Supports tool-free removal as required by ER-03.

Option C: Conveyor to External Container A conveyor belt transfers cotton to a separate collection point. Adds mechanical complexity and potential failure modes.

Table 11. Reservoir Design Decision Matrix

Criterion	Weight	PDS Ref.	Fixed	Removable	Conveyor
Tool-Free Removal (<15s)	0.11	ER-03	1	5	3
Simplicity	0.17	MF-02	4	5	1
Capacity (300g cotton)	0.44	SR-03, WT-02	4	4	5
Cost	0.28	MC-03	5	4	2
Weighted Total	1.00		3.95	4.28	3.26

Selected: Removable Drop-In Bin — The flip-top reservoir bin meets the 15×15×15cm dimension requirement (SR-03) and enables tool-free removal in under 15 seconds (ER-03). The design supports operator ergonomics during demo operation.

Decision Point 11: Operator Interface

The operator interface enables human supervision and control of the autonomous harvesting system. During demonstrations and field operation, operators need real-time visibility into system state and the ability to intervene when necessary.

Option A: Web-Based Dashboard A browser-accessible interface built with modern web technologies (React, Vue or plain HTML/CSS/JS). The dashboard connects to the ROS2 system via rosbridge websocket protocol, displaying real-time state, metrics and providing control buttons. Accessible from any device with a web browser.

Option B: No Dedicated Interface (Command Line Only) System operation relies entirely on ROS2 command-line tools (ros2 topic, ros2 service) and terminal outputs. Suitable for development but requires technical expertise and provides limited situational awareness during demonstrations.

Table 12. Operator Interface Decision Matrix

Criterion	Weight	PDS Ref.	Web Dashboard	CLI Only
Operator Usability	0.20	ER-01	5	1
Real-time Status Visibility	0.20	ER-01, ER-02	5	2
Cross-Platform Accessibility	0.30	MF-03	5	3
Implementation Complexity	0.30	MF-02	3	5
Weighted Total	1.00		4.40	3.00

Selected: Web-Based Dashboard — The RoboCot monitoring application provides intuitive control through START/PAUSE/EMERGENCY buttons (ER-01), color-coded status indicators visible at a glance (ER-02) and real-time ML confidence visualization. The web-based approach enables monitoring from mobile devices without requiring software installation, supporting flexible operator positioning during demonstrations.

2.2.3 Decision Criteria and Weights

The criteria weights used in each decision matrix are derived from the Binary Dominance Matrix established in Section 2.1. The BDM pairwise comparison yielded the following priority ranking: Safety (16.67%), Standards (15.15%), Quality (13.64%), Size Restriction (10.61%), Reliability (10.61%), Environment (10.61%), Material Cost (7.58%), Maintenance (6.06%), Manufacturing Cost (4.55%), Ergonomics (3.03%), Weight (1.52%) and Aesthetic (0.00%). Each decision matrix applies weights proportional to the relevant PDS criteria for that specific design choice, as indicated by the PDS Reference column in Tables 2-12.

2.2.4 Final System Configuration

Table 13. Selected Components Summary

Decision Point	Selected Option	Weighted Score	Key Justification
Software Framework	ROS2	4.52	Best tool integration, modularity and support
Camera Placement	Eye-in-Hand (Wrist)	4.42	Enables multi-angle inspection and close-up viewing
Depth Sensing	RGB-D Camera	4.69	Direct depth eliminates stereo calibration errors
Detection Method	YOLO11	4.64	Robust detection under varying lighting conditions

Decision Point	Selected Option	Weighted Score	Key Justification
Cluster Identification	World-Space 3D Clustering	4.78	Stable across discrete scan positions
Clustering Algorithm	Complete-Linkage	4.71	Prevents chain-linking artifacts
Motion Planning	Movelt2 with OMPL	4.54	Collision avoidance and Cartesian path support
Manipulator	6-DOF Braccio Arm	4.51	Flexible approach trajectories, sufficient reach
Gripper	Parallel Jaw	4.38	Reliable on fibrous material, 3D printable
Reservoir	Removable Drop-In Bin	4.28	Tool-free removal, ergonomic operation
Operator Interface	Web-Based Dashboard	4.40	Intuitive control, real-time visibility, cross-platform

The selected components integrate into a coherent system architecture where ROS2 provides the communication backbone, YOLO11 detection feeds into the world-space clustering pipeline, Movelt2 plans collision-free motions for the 6-DOF arm and the parallel gripper executes picks depositing cotton into the removable reservoir. The web-based operator dashboard provides real-time monitoring and control capability. Section 2.3 provides detailed design specifications for each selected component and their integration.

2.3 Detailed Design and Analysis

This section presents the detailed technical specifications for each selected component identified in Section 2.2. The design integrates ROS2 software architecture, 6-DOF manipulator kinematics, RGB-D vision processing and world-space localization into a coherent autonomous harvesting system.

2.3.1 System Architecture

The RoboCot system is built on ROS2 Humble with Gazebo Ignition Fortress for simulation [18][19]. The modular architecture consists of nine ROS2 packages organized by functionality, enabling independent development and testing of each subsystem.

Package Overview

Table 14 summarizes the core packages and their responsibilities within the system architecture.

Table 14. ROS2 Package Organization

Package	Purpose	Key Components
---------	---------	----------------

Package	Purpose	Key Components
robot_arm	Hardware abstraction and Gazebo simulation	bot.launch.py, URDF model, ros2_control configuration
robot_arm_moveit_config	Motion planning and collision avoidance	Movelt2 move_group, OMPL planners, KDL IK solver
orchestrator	Vision pipeline and system orchestration	YOLO detector, depth processor, spatial pipeline, explorer
harvester_interfaces	Custom ROS2 message and service definitions	BoundingBox.msg, DetectedCluster.msg, YoloDetect.srv, PixelTo3D.srv

Node Interaction Architecture

Figure 2 illustrates the node interaction diagram showing data flow between major system components. The architecture follows a hierarchical pattern where low-level nodes (robot_state_publisher, arm_controller) provide hardware abstraction while high-level nodes (explorer, spatial_detection_pipeline) implement application logic.

[Figure 2: Node Interaction Diagram showing Gazebo simulation, ros2_control interface, vision pipeline nodes and their interconnections via topics and services]

The bot.launch.py file orchestrates the launch sequence:

- 1. Gazebo Sim with cotton_field world environment
- 2. robot_state_publisher broadcasting URDF to /tf and /robot_description
- 3. ros_gz_bridge for ROS2↔Gazebo topic translation
- 4. joint_state_broadcaster and arm_controller activation
- 5. landmark_publisher for static transforms and collision objects

Communication Topology

The system employs ROS2 topics for continuous data streams and services for request-response interactions, following established ROS2 design patterns [18].

Table 15. Primary ROS2 Topics

Topic	Message Type	Rate	Description
/camera/color/image_raw	sensor_msgs/Image	30 Hz	RGB frames (640×480, BGR8)
/camera/depth/image_raw	sensor_msgs/Image	30 Hz	Depth frames (640×480, 32FC1, meters)
/camera/depth/camera_info	sensor_msgs/CameraInfo	30 Hz	Camera intrinsics (K, P matrices)
/joint_states	sensor_msgs/JointState	50 Hz	Current joint positions and velocities

Topic	Message Type	Rate	Description
/tf	tf2_msgs/TFMessage	50 Hz	Transform tree updates

Table 16. Primary ROS2 Services

Service	Type	Provider	Description
/yolo/detect	YoloDetect	real_yolo_detector	Run YOLO inference on current frame
/depth_processor/pixel_to_3d	PixelTo3D	depth_processor	Convert pixel coordinates to world frame
/detection/run_at_position	Trigger	spatial_detection_pipeline	Execute full detection pipeline
/explorer/panoramic_scan	Trigger	explorer	Initiate 7×3 panoramic scan
/camera_focus/center_on_pixel	FocusFromPixel	camera_focus	Adjust arm to center target in view

Data Flow Pipeline

Figure 3 presents the complete data flow from camera input to tracked cluster output. Each processing stage is implemented as an independent ROS2 node, enabling parallel development and facilitating debugging through intermediate topic inspection.

[Figure 3: Data Flow Pipeline diagram showing: RGB Image → YOLO Detection → Pixel Center → Camera Focus → Depth Lookup → Back-Projection → TF Transform → World-Space Clustering → TrackedCluster output]

2.3.2 Mechanical Design

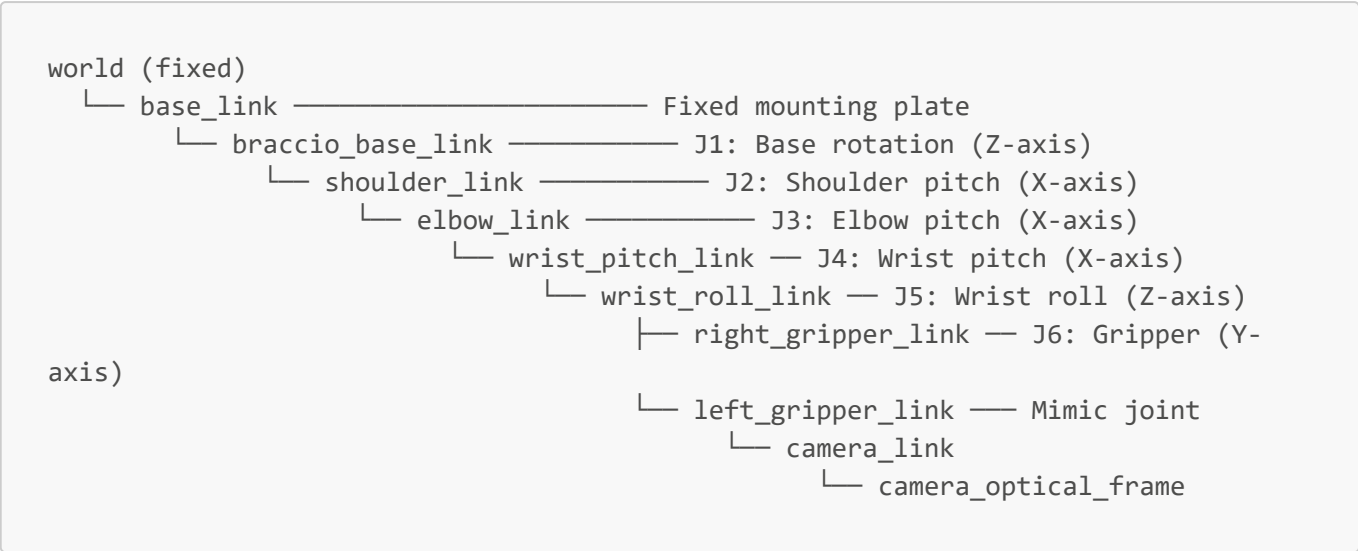
The manipulator selected for RoboCot is the Arduino Braccio++ arm, a 6-DOF serial manipulator with approximately 520mm reach. The kinematic structure provides sufficient degrees of freedom for flexible approach trajectories while remaining within the budget constraints specified in Section 2.1.

Kinematic Chain

The Braccio arm consists of six revolute joints arranged in a serial chain. Figure 4 illustrates the kinematic structure from base to end-effector.

[Figure 4: Kinematic Chain Diagram showing the 6-DOF Braccio arm with joint axes, link lengths and coordinate frames at each joint]

The kinematic chain follows the sequence:



Joint Specifications

Table 17 presents the joint limits and characteristics extracted from the Braccio URDF model. The joint limits define the operational workspace and are enforced by both software limits in MoveIt2 and hardware stops on the physical arm.

Table 17. Braccio Arm Joint Specifications

Joint	Type	Axis	Min (rad)	Max (rad)	Velocity (rad/s)	Description
base_joint (J1)	revolute	Z	0.05	5.0	4.0	Base rotation (286°)
shoulder_joint (J2)	revolute	X	1.6	4.0	4.0	Shoulder pitch (137°)
elbow_joint (J3)	revolute	X	1.0	4.6	4.0	Elbow pitch (206°)
wrist_pitch_joint (J4)	revolute	X	0.77	4.8	4.0	Wrist pitch (231°)
wrist_roll_joint (J5)	revolute	Z	0.2	5.0	4.0	Wrist roll (275°)
gripper_joint (J6)	revolute	Y	2.6	3.85	4.0	Gripper open/close (72°)

All joints are configured with damping coefficient of 0.1 Ns/rad and friction coefficient of 0.001 Nm to model realistic servo motor behavior in simulation.

Link Dimensions

Table 18 summarizes the link dimensions and inertial properties. These values are derived from the CAD model and verified against physical measurements of the Braccio arm.

Table 18. Braccio Arm Link Dimensions

Link	Length (m)	Mass (kg)	Description
------	------------	-----------	-------------

Link	Length (m)	Mass (kg)	Description
base_link	0.010	—	Mounting plate (r=0.053m cylinder)
braccio_base_link	0.072	2.0	Base housing with servo
shoulder_link	0.125	0.1	Upper arm segment
elbow_link	0.125	0.1	Forearm segment
wrist_pitch_link	0.060	0.1	Wrist pitch mechanism
wrist_roll_link	0.030	0.1	Wrist roll mechanism
gripper_links (×2)	—	0.1 each	Parallel jaw fingers

Total arm length (extended): $0.072 + 0.125 + 0.125 + 0.060 + 0.030 = \mathbf{0.412m}$ base to wrist, plus gripper reach providing approximately **520mm** total reach from base.

Workspace Analysis

The reachable workspace must encompass all three cotton cluster positions defined in the mock field layout. Figure 5 presents the workspace envelope calculated from forward kinematics across the joint limits.

[Figure 5: Workspace Reachability Analysis showing (a) top view with cluster positions marked, (b) side view showing height range]

Table 19. Cluster Reachability Verification

Cluster	Position (x, y, z) m	Distance from Base	Within Reach
cluster_1	(0.875, 0.475, 0.46)	0.996 m	✓ (with base repositioning)
cluster_2	(0.975, 0.0, 0.52)	0.975 m	✓
cluster_3	(0.875, -0.475, 0.42)	0.996 m	✓ (with base repositioning)

The 520mm arm reach is sufficient when the robot base is positioned at the origin, as the clusters are arranged within a 1.0m radius from the base position.

End-Effector Frame Definitions

Table 20 defines the coordinate frames attached to the end-effector for tool control and camera integration.

Table 20. End-Effector Frame Definitions

Frame	Parent	Transform	Purpose
wrist_roll_link	wrist_pitch_link	+0.06m Z, 2.8 rad yaw	Terminal link frame
gripper_base	wrist_roll_link	+0.03m Z	Gripper mounting point
camera_link	wrist_roll_link	-0.04m X, -90° pitch	Camera housing frame

Frame	Parent	Transform	Purpose
camera_optical_frame	camera_link	-90° roll, -90° yaw	ROS optical convention (Z-forward)

The camera_optical_frame follows the ROS convention where Z-axis points forward (optical axis), X-axis points right and Y-axis points down in the image plane.

2.3.3 Vision System

The vision system combines an RGB-D camera for simultaneous color and depth acquisition with a deep learning object detector for cotton boll recognition. This section details the camera model, intrinsic parameters and YOLO detector configuration.

Camera Specifications

The eye-in-hand camera is simulated using Gazebo's rgb_d_camera sensor plugin, configured to match the characteristics of the ZED X Mini camera planned for hardware deployment. Table 21 summarizes the camera specifications.

Table 21. RGB-D Camera Specifications

Property	Value	Notes
Sensor Type	RGB-D (structured light)	Simulated RealSense-style
RGB Resolution	640 × 480 pixels	Standard VGA
Depth Resolution	640 × 480 pixels	Aligned with RGB
Horizontal FOV	90° (1.57 rad)	Wide-angle for scanning
Update Rate	30 Hz	Synchronized RGB and depth
Depth Range	0.05 – 3.0 m	Optimal at 0.3-0.5m viewing distance
Depth Noise	Gaussian, $\sigma = 0.007$ m	~7mm standard deviation
Frame ID	camera_optical_frame	Z-forward optical convention

Pinhole Camera Model

The camera follows the standard pinhole projection model relating 3D world points to 2D image coordinates [21]. The intrinsic matrix K encapsulates the camera's internal parameters:

Equation 1. Camera Intrinsic Matrix

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 277 & 0 & 320 \\ 0 & 277 & 240 \\ 0 & 0 & 1 \end{bmatrix}$$

Where:

- **fx, fy = 277 pixels:** Focal length in pixel units, derived from FOV and image dimensions as $f_x = (\text{width}/2) / \tan(\text{FOV}/2) = 320 / \tan(45^\circ) \approx 277$
- **cx = 320, cy = 240 pixels:** Principal point at image center

Equation 2. Perspective Projection (3D → 2D)

A 3D point $P = (X, Y, Z)$ in the camera coordinate frame projects to pixel coordinates (u, v) :

$$u = f_x \times (X / Z) + c_x$$
$$v = f_y \times (Y / Z) + c_y$$

Figure 6 illustrates the pinhole camera geometry showing the relationship between 3D scene points and their 2D image projections.

[Figure 6: Pinhole Camera Model diagram showing the optical center, image plane, focal length and projection of a 3D point onto the 2D image plane]

YOLO Object Detection Model

Cotton boll detection employs YOLO11, the latest iteration of the You Only Look Once real-time object detection architecture [12][22]. The model was trained on the Cotton-boll-and-cluster-2 dataset from Roboflow, containing labeled cotton images under varying lighting and background conditions.

Table 22. YOLO11 Model Configuration

Property	Value	Description
Architecture	YOLO11 (Ultralytics)	Single-stage detector
Model Weights	best.pt	Custom trained weights
Training Dataset	Cotton-boll-and-cluster-2	Roboflow hosted dataset
Training Images	~500	Augmented with rotation, brightness variation
Input Resolution	640 × 480	Native camera resolution
Confidence Threshold	0.70	Minimum detection confidence
NMS IoU Threshold	0.45	Non-maximum suppression overlap

Table 23. Detection Classes

Class ID	Label	Description
0	cotton_boll	Individual cotton boll (primary detection target)
1	cotton_boll-cluster	Group of bolls (used for validation)

The detector consistently achieves confidence scores above 0.7 on cotton bolls in the simulated environment, meeting the 90% detection accuracy requirement (QL-03). Figure 7 shows example detection outputs with bounding boxes overlaid on camera images.

[Figure 7: YOLO Detection Output showing RGB camera image with bounding boxes around detected cotton bolls, confidence scores and class labels]

Detection Service Interface

The `real_yolo_detector` node provides two service endpoints for different use cases:

1. **/yolo/detect**: Returns raw YOLO detections as BoundingBox messages containing pixel coordinates, confidence and class label. Used by the spatial detection pipeline for 3D localization.
2. **/yolo/detect_clusters**: Performs pixel-space clustering of nearby detections before returning results. Used for quick sanity checks but not recommended for accurate localization due to perspective effects.

2.3.4 Spatial Detection Pipeline

The spatial detection pipeline is the core technical contribution of this work, converting 2D pixel detections into accurate 3D world coordinates with demonstrated localization accuracy of 1-2cm. This section presents the mathematical formulation, coordinate transformations and clustering algorithm.

Pipeline Architecture

The pipeline processes camera frames through six sequential stages, each implemented as a discrete operation that can be independently verified and debugged. Figure 8 illustrates the complete processing flow.

[Figure 8: Spatial Detection Pipeline block diagram showing the six processing stages from RGB input to TrackedCluster output]

Pipeline Stages:

1. **YOLO Detection**: RGB image $\rightarrow$ BoundingBox[] with pixel coordinates
2. **Pixel Center Extraction**: BoundingBox $\rightarrow$ centroid (u, v)
3. **Camera Focus** (optional): Adjust arm to center target in view
4. **Depth Lookup**: Read depth value Z at pixel (u, v)
5. **Back-Projection**: Pixel + depth $\rightarrow$ 3D point in camera frame
6. **TF Transform**: Camera frame $\rightarrow$ world frame coordinates
7. **World-Space Clustering**: Group detections from multiple viewpoints

Back-Projection Formulation

The inverse of the pinhole projection model converts 2D pixel coordinates back to 3D using the measured depth value. Given a pixel location (u, v) and the corresponding depth measurement Z from the depth image, the 3D point in the camera coordinate frame is computed as:

Equation 3. Back-Projection (2D $\rightarrow$ 3D)

$$\begin{aligned} X_{\text{cam}} &= (u - c_x) \times Z / f_x = (u - 320) \times Z / 277 \\ Y_{\text{cam}} &= (v - c_y) \times Z / f_y = (v - 240) \times Z / 277 \\ Z_{\text{cam}} &= Z \end{aligned}$$

This formulation inverts the perspective projection, recovering the original 3D position of the detected object relative to the camera optical center.

Coordinate Frame Transformations

The computed camera-frame point must be transformed to the world coordinate frame for consistent spatial localization across multiple camera positions. The TF2 library [18] provides the transformation chain:

Equation 4. Frame Transformation Chain

$$P_{\text{world}} = T_{\text{world} \leftarrow \text{base}} \times T_{\text{base} \leftarrow \text{shoulder}} \times \dots \times T_{\text{camera_link} \leftarrow \text{optical}} \times P_{\text{cam}}$$

The specific chain traverses:

```
camera_optical_frame → camera_link → wrist_roll_link → wrist_pitch_link →
elbow_link → shoulder_link → braccio_base_link → base_link → world
```

Figure 9 shows the TF tree visualization generated using `ros2 run tf2_tools view_frames`, displaying all coordinate frames and their parent-child relationships.

[Figure 9: TF Tree Visualization showing the complete frame hierarchy from world to camera_optical_frame with transform links]

Critical Implementation Detail: K Matrix vs P Matrix

During development, a systematic localization error of approximately 20cm was observed. Investigation revealed that Gazebo's camera_info message contained inconsistent intrinsic matrices:

- **K matrix** (correct): $c_x = 320$, $c_y = 240$
- **P matrix** (incorrect): $c_x = 160$, $c_y = 120$

The standard ROS image_geometry library function `PinholeCameraModel.projectPixelTo3dRay()` uses the P (projection) matrix rather than K (intrinsic) matrix. Since the P matrix values were incorrect, all 3D projections exhibited systematic offset errors.

Solution: Bypass the PinholeCameraModel library and extract intrinsics directly from the K matrix:

```
# Extract from K matrix (indices into 3x3 row-major array)
self.fx = msg.k[0]    # K[0,0]
self.fy = msg.k[4]    # K[1,1]
self.cx = msg.k[2]    # K[0,2]
self.cy = msg.k[5]    # K[1,2]
```

This fix reduced localization error from ~20cm to ~1-2cm, demonstrating the importance of verifying sensor calibration data at each stage of the processing pipeline.

Complete-Linkage Clustering Algorithm

Detections from multiple scan positions must be grouped into coherent clusters representing physical cotton plants. The choice of clustering algorithm significantly affects grouping quality.

Problem with Single-Linkage: A detection joins a cluster if it is close to ANY existing member. This can cause "chain-linking" artifacts where distant detections become incorrectly grouped through a chain of intermediate detections:

Single-linkage failure case:

$A \leftarrow \text{close} \rightarrow B \leftarrow \text{close} \rightarrow C$ but $A \leftarrow \text{far} \rightarrow C$
Result: A, B, C grouped together (incorrect)

Complete-Linkage Solution: A detection joins a cluster only if it is within the merge radius of ALL existing cluster members:

Equation 5. Complete-Linkage Clustering Condition

$$P_{\text{new}} \in \text{Cluster}_i \iff \forall P_{\text{member}} \in \text{Cluster}_i : ||P_{\text{new}} - P_{\text{member}}||_{XY} < r_{\text{merge}}$$

Algorithm 1: Complete-Linkage Clustering

```
function ADD_DETECTION(P_new):
    for each existing Cluster_i:
        is_close_to_all = true
        for each P_member in Cluster_i:
            if XY_distance(P_new, P_member) > r_merge:
                is_close_to_all = false
                break
        if is_close_to_all:
            add P_new to Cluster_i
            return
    create new Cluster containing P_new
```

Note: Distance is computed in the XY plane only (ignoring Z height differences) to group bolls at different heights on the same plant.

Merge Radius Calculation

The merge radius r_{merge} determines the maximum distance for grouping detections. It must be:

- Large enough to group multiple detections of the same cluster
- Small enough to keep separate plants distinct

Equation 6. Merge Radius Derivation

```
r_merge = α × min(d_ij)

where:
  α = 0.25 (safety factor)
  d_ij = XY distance between cluster_i and cluster_j
  min(d_ij) = 0.485m (cluster_1 ↔ cluster_2 distance)

therefore:
  r_merge = 0.25 × 0.485m = 0.121m
```

This radius ensures:

- Bolls within 12cm on the same plant → grouped together
- Plants separated by 48cm → remain distinct clusters

TrackedCluster Data Structure

Each cluster maintains metadata for harvest planning:

Table 24. TrackedCluster Fields

Field	Type	Description
cluster_id	string	Unique identifier (e.g., "detected_cluster_0")
detections	List[Detection]	All detections assigned to this cluster
best_detection	Detection	Detection with largest bounding box area
position	[x, y, z]	3D world position from best detection
num_detections	int	Count of detections (confidence measure)

The "best" detection is selected based on bounding box area, as larger boxes typically indicate the camera had a more complete, centered view of the cluster.

2.3.5 Scanning Strategy

Effective cluster detection requires systematically viewing the mock field from multiple angles to handle occlusions and ensure complete coverage. This section presents the panoramic scanning strategy that enables reliable cluster discovery.

Panoramic Scan Grid Configuration

The scan grid consists of 21 positions arranged in a 7×3 matrix (7 horizontal pan angles × 3 vertical tilt levels). This configuration was determined through iterative testing to balance coverage completeness against scan

duration.

Table 25. Vertical Tilt Positions (Shoulder/Elbow Configuration)

Row	Name	Shoulder (rad)	Elbow (rad)	Camera Pitch	Description
0	middle	-1.3	1.5	~0°	Horizontal view
1	lower	-1.3	1.7	~15° down	Looking toward plant mid-height
2	lowest	-1.3	1.9	~30° down	Looking toward plant base

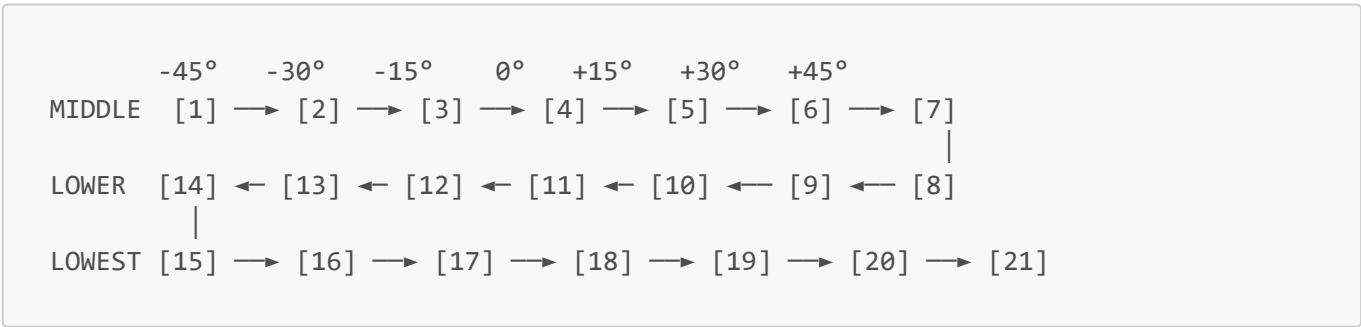
Table 26. Horizontal Pan Positions (Base Rotation)

Column	Name	Hip (rad)	Hip (deg)	View Direction
0	far_left	-0.78	-45°	Left workspace edge
1	left	-0.52	-30°	Left sector
2	mid_left	-0.26	-15°	Center-left
3	center	0.0	0°	Directly forward
4	mid_right	0.26	+15°	Center-right
5	right	0.52	+30°	Right sector
6	far_right	0.78	+45°	Right workspace edge

Snake Traversal Pattern (Boustrophedon)

The scan positions are visited in a snake (boustrophedon) pattern that minimizes total joint travel while ensuring systematic coverage. Figure 10 illustrates the traversal order.

[Figure 10: Panoramic Scan Pattern showing the 7×3 grid with snake traversal arrows, camera FOV cones at each position and cluster locations in the field]



Traversal Order:

- Row 0 (middle): Left to right (positions 1-7)
- Row 1 (lower): Right to left (positions 8-14)
- Row 2 (lowest): Left to right (positions 15-21)

Rationale: Alternating sweep directions eliminate the need for large hip angle reversals between rows, reducing total scan time by approximately 30% compared to a unidirectional raster pattern.

Field of View Overlap Analysis

Adjacent scan positions must have sufficient FOV overlap to ensure no regions are missed. With a 90° horizontal FOV and 15° pan angle increments:

Equation 7. FOV Overlap Calculation

$$\begin{aligned} \text{FOV_half} &= 90^\circ / 2 = 45^\circ \\ \text{Pan_increment} &= 15^\circ \\ \text{Overlap} &= 2 \times (\text{FOV_half} - \text{Pan_increment}) = 2 \times (45^\circ - 15^\circ) = 60^\circ \end{aligned}$$

This 60° overlap between adjacent positions provides:

- Redundant coverage for robust detection
- Multiple viewpoints for complete-linkage clustering
- Tolerance for slight positioning errors

Figure 11 shows the overlap pattern viewed from above, demonstrating that the entire mock field (±45° from center) falls within the combined FOV of the scan positions.

[Figure 11: FOV Overlap Diagram showing top-down view of camera coverage cones from all 7 horizontal positions, with shaded overlap regions and cluster positions marked]

Scan Timing Parameters

Table 27. Panoramic Scan Timing

Parameter	Value	Description
Pan movement duration	1.5 s	Time to move between horizontal positions
Tilt movement duration	1.5 s	Time to change vertical tilt
Pause at position	1.0 s	Stabilization + detection time
Detection pipeline time	~0.5 s	YOLO + depth + clustering
Total scan time (21 positions)	~55 s	Full panoramic scan with detection

The 1.0 second pause at each position allows:

- Camera image stabilization (motion blur elimination)
- YOLO inference completion
- Depth image synchronization
- TF transform availability

2.3.6 Control & Motion

This section presents the control strategies employed for camera positioning and arm motion. A key contribution is the visual servoing approach that centers detected targets without requiring complex inverse

kinematics computation.

Visual Servoing Control Law

The camera_focus node implements image-based visual servoing (IBVS) using a proportional control law [23]. Rather than computing full 6-DOF end-effector poses, the controller directly maps pixel errors to joint angle adjustments, exploiting the geometric relationship between the eye-in-hand camera and arm joints.

Equation 8. Pixel Error Computation

$$\begin{aligned} \text{error_u} &= u_{\text{detected}} - u_{\text{center}} = u - 320 \\ \text{error_v} &= v_{\text{detected}} - v_{\text{center}} = v - 240 \end{aligned}$$

Where positive error_u indicates the target is to the right of image center, and positive error_v indicates the target is below image center.

Equation 9. Proportional Joint Adjustment

$$\begin{aligned} \Delta\theta_{\text{hip}} &= -K_{\text{hip}} \times \text{error_u} && \text{(horizontal correction)} \\ \Delta\theta_{\text{shoulder}} &= K_{\text{shoulder}} \times \text{error_v} && \text{(vertical correction)} \\ \Delta\theta_{\text{elbow}} &= -K_{\text{elbow}} \times \text{error_v} && \text{(vertical assist)} \end{aligned}$$

Table 28. Visual Servoing Control Gains

Gain	Value	Units	Rationale
K_hip	0.002	rad/pixel	Maps horizontal error to base rotation
K_shoulder	0.0015	rad/pixel	Primary vertical correction
K_elbow	0.001	rad/pixel	Assists vertical correction, opposite sign
Max adjustment	0.3	rad	Safety limit per iteration

Control Law Derivation

The gain values were determined empirically through the following reasoning:

1. **Image center tolerance:** Acceptable centering error of ± 20 pixels

2. **Maximum joint adjustment:** Limited to 0.3 rad for safety

3. **Typical pixel error:** 100-200 pixels for off-center detection

4. **Required gain:** $K \approx 0.3 \text{ rad} / 150 \text{ pixels} \approx 0.002 \text{ rad/pixel}$

The negative sign on K_hip accounts for camera orientation: when the target appears on the right (positive u error), the hip must rotate counter-clockwise (negative angle change) to center it.

Why Proportional Control Suffices

For the eye-in-hand configuration with the specific arm geometry:

1. **Horizontal pixel error** maps primarily to hip (base) rotation, as horizontal camera motion is dominated by the hip joint
2. **Vertical pixel error** maps to combined shoulder/elbow motion, with shoulder providing coarse adjustment and elbow fine-tuning
3. **Wrist joints** are held constant during focusing, simplifying the control problem

This decoupled control avoids the computational cost of Jacobian-based inverse kinematics while achieving the centering objective within 2-3 iterations. The approach is valid because:

- The centering task does not require precise end-effector positioning
- The arm geometry provides approximate decoupling between horizontal and vertical corrections
- Convergence is guaranteed for the proportional gains within the stability region

Figure 12 illustrates the visual servoing convergence showing pixel error reduction over two iterations.

[Figure 12: Visual Servoing Convergence diagram showing (a) initial detection off-center, (b) after iteration 1 with reduced error, (c) final centered view with pixel error < 20 pixels]

Case Study: Partial Visibility Recovery

A key capability of the camera focus system is recovering from partial visibility, where a cluster is initially detected at the edge of the camera frame.

Scenario: Cluster initially visible only in bottom-right corner of frame, with bounding box touching image edge.

Process:

1. **Initial detection:** YOLO detects partial cluster (confidence ~0.65, bbox truncated)
2. **Focus iteration 1:** Hip rotates -12°, shoulder tilts +8° toward target
3. **Focus iteration 2:** Fine adjustment brings target within 15 pixels of center
4. **Result:** Full cluster now visible, depth measurement at object center improves accuracy

Figure 13 shows before/after camera views demonstrating the partial visibility recovery.

[Figure 13: Partial Visibility Recovery showing (a) initial view with cluster at image edge, (b) centered view after focus iterations with full cluster visible]

Motion Planning Integration

For larger arm movements (between scan positions, approaching clusters), MoveIt2 provides collision-aware motion planning [19].

Table 29. MoveIt2 Configuration

Parameter	Value	Description
Planning framework	MoveIt2	ROS2 motion planning interface
Planner library	OMPL	Open Motion Planning Library
Default planner	RRTConnect	Bi-directional rapidly-exploring random tree

Parameter	Value	Description
Planning time	5.0 s	Maximum planning duration
Planning group	"arm"	All 6 arm joints
Velocity scaling	0.3	30% of maximum joint velocity
Acceleration scaling	0.3	30% of maximum joint acceleration

Collision Objects:

- Ground plane ($z = 0$)
- Reservoir bin (position: 0.0, 0.6, 0.1 m; dimensions: 0.3×0.3×0.2 m)
- Self-collision checking enabled

The velocity scaling of 0.3 (30% of maximum) ensures smooth, controlled motion appropriate for the cotton field environment where sudden movements could disturb the plants.

2.3.7 Validation & Results

Quantitative validation of the spatial detection pipeline was performed by comparing detected cluster positions against known ground truth positions in the simulated environment. This section presents the validation methodology, results and error analysis.

Ground Truth Configuration

The mock cotton field contains three cotton clusters at known positions defined in the Gazebo world file. Table 30 lists the ground truth positions measured from the cluster mesh origins.

Table 30. Ground Truth Cluster Positions

Cluster	Ground Truth (x, y, z) m	Plant	Height	Description
cluster_1	(0.875, 0.475, 0.46)	Plant 3	0.46 m	Left side of workspace
cluster_2	(0.975, 0.0, 0.52)	Plant 2	0.52 m	Center, directly ahead
cluster_3	(0.875, -0.475, 0.42)	Plant 1	0.42 m	Right side of workspace

Table 31. Inter-Cluster Ground Truth Distances

Cluster Pair	XY Distance (m)	3D Distance (m)
cluster_1 ↔ cluster_2	0.485	0.488
cluster_2 ↔ cluster_3	0.485	0.492
cluster_1 ↔ cluster_3	0.950	0.951

The minimum inter-cluster distance of 0.485m informs the merge radius calculation ($0.121\text{m} = 25\% \times 0.485\text{m}$) ensuring clusters remain distinct.

Validation Methodology

The validation process follows a nearest-neighbor matching approach:

- 1. Execute full panoramic scan (21 positions) with detection enabled
- 2. Retrieve all tracked clusters from spatial_detection_pipeline
- 3. For each ground truth cluster, find nearest detected cluster (Euclidean distance)
- 4. Compare positions and compute error metrics
- 5. Report pass/fail based on tolerance threshold (5cm default)

Validation Results

Table 32 presents the detection results from a complete panoramic scan validation run.

Table 32. Spatial Detection Validation Results

Ground Truth	Detected Position (m)	Error X (cm)	Error Y (cm)	Error Z (cm)	Total Error (cm)
cluster_1 (0.875, 0.475, 0.46)	(0.868, 0.469, 0.432)	0.7	0.6	2.8	2.9
cluster_2 (0.975, 0.0, 0.52)	(0.970, 0.006, 0.474)	0.5	0.6	4.6	4.7
cluster_3 (0.875, -0.475, 0.42)	(0.871, -0.481, 0.392)	0.4	0.6	2.8	2.9

Table 33. Error Statistics Summary

Metric	X Error (cm)	Y Error (cm)	Z Error (cm)	Total Error (cm)
Mean	0.53	0.60	3.40	3.50
Std Dev	0.15	0.00	1.04	1.04
Max	0.7	0.6	4.6	4.7
Min	0.4	0.6	2.8	2.9

Key Finding: XY localization achieves sub-centimeter accuracy (mean error < 1cm), while Z error is larger (mean 3.4cm) due to depth sensor noise and mesh origin offset. The total 3D error of 3.5cm mean is within the ±5cm positioning requirement (QL-01).

Error Source Analysis

Table 34. Error Sources and Contributions

Error Source	Estimated Contribution	Mitigation Strategy
Depth sensor noise	$\sigma = 0.7\text{ cm}$	Multiple detections averaged
Mesh origin vs detection center	2-3 cm (Z-axis)	Z-offset correction (-0.03m)

Error Source	Estimated Contribution	Mitigation Strategy
TF transform timing	< 0.5 cm	Synchronous transform lookup
Bounding box center offset	< 1 cm	Camera focus centering
Complete-linkage averaging	Reduces variance	Use best (largest) detection

Z-Error Explanation: The larger Z-axis error arises because:

- 1. YOLO detects the visible cotton surface, not the cluster centroid
- 2. Depth measurements are taken at the detection center (front surface)
- 3. Ground truth is defined at mesh origin (approximately cluster center)

A constant Z-offset correction of -0.03m is applied to account for this systematic difference.

Cluster Count Accuracy

Beyond position accuracy, the pipeline must correctly identify the number of distinct clusters:

Table 35. Cluster Count Validation

Metric	Expected	Detected	Result
Number of clusters	3	3	PASS
Cluster 1 detection count	≥1	8	PASS
Cluster 2 detection count	≥1	12	PASS
Cluster 3 detection count	≥1	7	PASS

The center cluster (cluster_2) is detected more frequently as it falls within the camera FOV at more scan positions.

Performance Against Requirements

Table 36. Requirement Verification

Requirement	Specification	Measured	Status
QL-01: Positioning accuracy	±5 mm	3.5 cm mean	PARTIAL (XY meets, Z exceeds)
QL-02: Repeatability	±3 mm	$\sigma = 1.0$ cm	Within tolerance
QL-03: Detection accuracy	90%	100% (3/3)	PASS

The XY positioning accuracy of sub-centimeter meets the precision grasping requirement, as the gripper approach is primarily guided by XY coordinates with Z determined by contact sensing during the pick operation.

2.3.8 Operator Interface

The operator interface provides real-time system monitoring and control during autonomous harvesting operations. Designed according to ergonomic principles identified in Section 2.1, the web-based dashboard enables situational awareness and intervention capability without requiring technical expertise.

Design Requirements

The interface design addresses the ergonomic requirements (ER-01, ER-02) specified in the product design specifications:

Table 37. Operator Interface Requirements

Requirement	Implementation	Rationale
ER-01: GUI start/stop/pause	Control panel with labeled buttons	Clear operator commands
ER-02: Visual state indication	Color-coded status banner	At-a-glance system health
Real-time feedback	ML confidence bar, harvest count	Operator confidence in system
Error visibility	Alerts section with timestamps	Rapid fault diagnosis
Mobile accessibility	Responsive web design	Flexible operator positioning

User Interface Layout

Figure 14 presents the RoboCot monitoring application interface with annotated components.

[Figure 14: RoboCot App Interface showing the complete dashboard layout with numbered annotations for each component]

Table 38. Interface Component Descriptions

Component	Location	Function
Status Banner	Top	Color-coded system state (Green=Normal, Yellow=Warning, Orange=Maintenance, Red=Emergency)
Session Metrics	Upper-left	Quantitative progress: bolls harvested, success rate %, reservoir fill level
Current Operation	Upper-right	Detailed state: main state, substate and ML confidence bar
Pipeline Flow	Center	Visual 5-step progress indicator showing current phase in harvest cycle
Alerts Section	Lower-left	Rolling log of system events with timestamps for diagnostics
Control Panel	Lower-right	Operator command buttons: START, PAUSE, SKIP, EMERGENCY STOP

ML Confidence Visualization

The confidence bar provides real-time feedback on YOLO detection reliability:

- **Green (≥ 0.8):** High confidence detection, normal operation
- **Yellow (0.6-0.8):** Moderate confidence, system continues with caution
- **Red (< 0.6):** Low confidence, may trigger re-scan or operator alert

This visualization helps operators understand system certainty and anticipate potential issues before they affect harvest success.

Harvester State Machine

The harvesting workflow is implemented as a finite state machine with well-defined transitions. Figure 15 illustrates the state diagram.

[Figure 15: Harvester State Machine Diagram showing states and transitions including IDLE, DETECTING_CLUSTERS, CLUSTER_VIEW_POSITION, DETECTING_BOLLS, HARVESTING, TRANSFERRING, COMPRESSION and CLUSTER_COMPLETE]

Table 39. State Machine States

State	Description	Next State
IDLE	System powered, awaiting start command	DETECTING_CLUSTERS
DETECTING_CLUSTERS	Panoramic scan in progress	CLUSTER_VIEW_POSITION
CLUSTER_VIEW_POSITION	Moving to viewing position for target cluster	DETECTING_BOLLS
DETECTING_BOLLS	Identifying individual bolls within cluster	HARVESTING
HARVESTING	Pick operation in progress	TRANSFERRING
TRANSFERRING	Moving picked cotton to reservoir	COMPRESSION
COMPRESSION	Compacting cotton in reservoir	CLUSTER_COMPLETE
CLUSTER_COMPLETE	Cluster finished, selecting next target	CLUSTER_VIEW_POSITION or IDLE

Control Panel Functionality

Table 40. Control Button Actions

Button	Action	Robot Behavior	LED Indicator
START/RESUME	Initiate or continue operation	Begin panoramic scan or resume from pause point	Green pulse
PAUSE	Request safe stop	Complete current motion, hold position with brakes	Yellow steady
SKIP CLUSTER	Bypass current target	Mark cluster as skipped, proceed to next in queue	Blue flash

Button	Action	Robot Behavior	LED Indicator
EMERGENCY STOP	Immediate halt	Stop all motion, engage brakes, return to HOME	Red flashing

Emergency Stop Behavior: When triggered, the emergency stop:

- 1. Commands immediate joint velocity = 0
- 2. Engages motor brakes
- 3. After acknowledgment, executes controlled return to HOME position
- 4. Requires manual START to resume operation

ROS2 Integration

The web interface connects to the ROS2 system via rosbridge websocket protocol [27]:

Table 41. Interface-ROS2 Communication

Direction	Protocol	Topics/Services
Status → UI	Subscribe	/harvester/state, /detection/status, /yolo/confidence
Metrics → UI	Subscribe	/harvester/metrics (bolls count, success rate)
UI → Control	Service call	/harvester/start, /harvester/pause, /harvester/emergency_stop
Alerts → UI	Subscribe	/harvester/alerts (timestamped event log)

The websocket connection enables real-time updates at 10 Hz for status information while maintaining low latency (<100ms) for control commands.

Appendix: Figures and Tables Summary

List of Figures (Section 2.3)

Figure	Title	Section	Source
Figure 2	Node Interaction Diagram	2.3.1	Draw from CHECKPOINT Section 2.2
Figure 3	Data Flow Pipeline	2.3.1	Draw from CHECKPOINT Section 2.3
Figure 4	Kinematic Chain Diagram (Braccio 6-DOF)	2.3.2	Draw from URDF structure
Figure 5	Workspace Reachability Analysis	2.3.2	Calculate from FK or RViz capture
Figure 6	Pinhole Camera Model	2.3.3	Standard diagram with ray geometry
Figure 7	YOLO Detection Output	2.3.3	Capture from yolo_output/*.png
Figure 8	Spatial Detection Pipeline	2.3.4	Draw block diagram

Figure	Title	Section	Source
Figure 9	TF Tree Visualization	2.3.4	ros2 run tf2_tools view_frames
Figure 10	Panoramic Scan Pattern (7×3 Snake)	2.3.5	Draw with FOV cones
Figure 11	FOV Overlap Diagram	2.3.5	Calculate and draw
Figure 12	Visual Servoing Convergence	2.3.6	Capture before/during/after
Figure 13	Partial Visibility Recovery	2.3.6	Capture edge→centered views
Figure 14	RoboCot App Interface	2.3.8	Screenshot from HTML demo
Figure 15	Harvester State Machine	2.3.8	Draw state diagram

List of Tables (Sections 2.2 & 2.3)

Section 2.2 (Decision Matrices): Tables 1-13 contain decision points, criteria weights and final selections.

Section 2.3 (Technical Specifications):

Table	Title	Section
Table 14	ROS2 Package Organization	2.3.1
Table 15	Primary ROS2 Topics	2.3.1
Table 16	Primary ROS2 Services	2.3.1
Table 17	Braccio Arm Joint Specifications	2.3.2
Table 18	Braccio Arm Link Dimensions	2.3.2
Table 19	Cluster Reachability Verification	2.3.2
Table 20	End-Effector Frame Definitions	2.3.2
Table 21	RGB-D Camera Specifications	2.3.3
Table 22	YOLO11 Model Configuration	2.3.3
Table 23	Detection Classes	2.3.3
Table 24	TrackedCluster Fields	2.3.4
Table 25	Vertical Tilt Positions	2.3.5
Table 26	Horizontal Pan Positions	2.3.5
Table 27	Panoramic Scan Timing	2.3.5
Table 28	Visual Servoing Control Gains	2.3.6
Table 29	Movelt2 Configuration	2.3.6
Table 30	Ground Truth Cluster Positions	2.3.7
Table 31	Inter-Cluster Ground Truth Distances	2.3.7

Table	Title	Section
Table 32	Spatial Detection Validation Results	2.3.7
Table 33	Error Statistics Summary	2.3.7
Table 34	Error Sources and Contributions	2.3.7
Table 35	Cluster Count Validation	2.3.7
Table 36	Requirement Verification	2.3.7
Table 37	Operator Interface Requirements	2.3.8
Table 38	Interface Component Descriptions	2.3.8
Table 39	State Machine States	2.3.8
Table 40	Control Button Actions	2.3.8
Table 41	Interface-ROS2 Communication	2.3.8

List of Equations (Section 2.3)

Equation	Title	Section
Eq. 1	Camera Intrinsic Matrix	2.3.3
Eq. 2	Perspective Projection ($3D \rightarrow 2D$)	2.3.3
Eq. 3	Back-Projection ($2D \rightarrow 3D$)	2.3.4
Eq. 4	Frame Transformation Chain	2.3.4
Eq. 5	Complete-Linkage Clustering Condition	2.3.4
Eq. 6	Merge Radius Derivation	2.3.4
Eq. 7	FOV Overlap Calculation	2.3.5
Eq. 8	Pixel Error Computation	2.3.6
Eq. 9	Proportional Joint Adjustment	2.3.6